AION: A Closed-Loop, Human-Gated Framework for LLM-Assisted Incident Detection, Root-Cause Analysis, and Validated Remediation

Aditya Bheke
Dept. of Computer Engineering
Ajeenkya D Y Patil School of Engineering
Pune, India
aditya.bheke@dypic.in

Harshal Bhogawade
Dept. of Computer Engineering
Ajeenkya D Y Patil School of Engineering
Pune, India
harshal.bhogawade@dypic.in

Malhar Bhoi
Dept. of Computer Engineering
Ajeenkya D Y Patil School of Engineering
Pune, India
malhar.bhoi@dypic.in

Dr. Yogesh Bahendwar
Dept. of Computer Engineering
Ajeenkya D Y Patil School of Engineering
Pune, India

***Abstract*—Resolving a production incident still requires engineers to search large volumes of logs, trace the failure to the code change that caused it, write and test a fix, and deploy it under time pressure. Prior work automates individual stages of this process—log parsing and anomaly detection, root-cause analysis (RCA), or automated program repair—but the stages are usually studied in isolation, and repair techniques are evaluated on benchmark bugs disconnected from live telemetry, deployment history and release pipelines. We present AION (Autonomous Incident Observation & Navigation), a framework that connects these stages in a single closed loop while keeping a human in control of production. AION deduplicates structured logs into error signatures, detects incidents as signature-level error spikes, ranks suspect commits with an explainable weighted score that combines `git blame`, function scope, dependency overlap and deployment timing, and asks a large language model (LLM) for an evidence-cited diagnosis that is checked against a curated evidence pack. Candidate patches are generated on isolated branches under a code-enforced policy and are accepted only after a deterministic validation pipeline (fail-to-pass regression test, unit tests, staging deployment and replay of the failing requests). Deployment is possible only through an explicit human approval that is bound to the exact validated commit. On four seeded scenarios with known root causes, AION detected every incident with no false positive and ranked the true culprit first in all three faulty releases; with a locally hosted 7B-parameter model, its LLM diagnosis named the culprit in two of three cases, and every incorrect AI patch was rejected by validation before reaching the approval gate.**

***Index Terms*—AIOps, incident management, log analysis, root-cause analysis, large language models, automated program repair, human-in-the-loop, continuous deployment**

I. Introduction

Modern software services produce continuous streams of logs, alerts and deployment events. When a release introduces a fault, on-call engineers must find the relevant error among thousands of log lines, decide which recent change caused it, write and test a fix, and push that fix through the deployment pipeline. Empirical studies of cloud incident management report that this process remains expensive and dependent on experienced engineers, even in organisations with mature tooling [1], [2].

Research in AIOps has made substantial progress on each stage separately. Log parsers such as Drain [3] and LILAC [4] convert raw logs into templates; DeepLog [5], LogAnomaly [6], NeuralLog [7] and BERT-based detectors [8]–[10] detect anomalous log behaviour. RCA techniques localise faulty services [11]–[13], and recent work shows that LLMs can recommend root causes and mitigations for real cloud incidents [14]–[16]. In parallel, LLM-based automated program repair (APR) and software engineering agents can generate and test code fixes [17]–[20].

Two gaps remain. First, these stages are rarely connected: detection and RCA systems stop at a diagnosis, while repair systems start from a bug report or a failing test and are evaluated on benchmarks such as Defects4J or SWE-bench [21] rather than on live production signals, deployment history and release pipelines. Recent surveys identify this fragmentation, together with hallucination and unreliable evaluation, as open problems for LLM-based AIOps [22], [23]. Second, letting an LLM change production code is unsafe without guarantees: a plausible but wrong fix can make an incident worse, and work on LLM-based remediation explicitly calls for safety checks [24].

AION addresses both gaps with a closed-loop workflow in which AI may *investigate, analyse, propose and validate*, but only a human may *approve* a production change. This paper makes the following contributions:

- An end-to-end architecture that connects log-based detection, change correlation, LLM diagnosis, patch gener-

ation, deterministic validation and deployment, with the human approval gate enforced structurally by a finite-state machine and bound to a commit hash (Section III).
- An explainable change-correlation score that combines line- and function-level `git blame`, file and dependency overlap and deployment timing, extending the classical idea of fix-inducing changes [25] to online incident triage (Section IV-C).
- An evidence-pack and grounding scheme that requires every LLM claim to cite a verifiable evidence item and caps confidence when claims cannot be verified (Section IV-D).
- A policy-constrained generate–validate–repair loop with fail-to-pass checking, staging request replay and a revert fallback (Sections IV-E–IV-F).
- A scenario-based evaluation with known answers that measures each stage separately and documents how the evaluation exposed and fixed four weaknesses of the initial design (Section VI).

II. Related Work

A. Log Parsing and Anomaly Detection

Surveys of automated log analysis cover the full pipeline from logging practice through parsing, anomaly detection, failure prediction and diagnosis [26], and public datasets such as Loghub [27] have enabled systematic benchmarking. Drain [3] parses logs online with a fixed-depth tree; LILAC [4] uses LLMs with an adaptive parsing cache to improve template accuracy while reducing LLM queries. DeepLog [5] and LogAnomaly [6] learn normal log sequences; NeuralLog [7] avoids parsing errors by embedding raw messages; LogBERT [8] and LAnoBERT [9] apply self-supervised BERT models, and a recent industrial deployment runs a LogBERT model locally for confidentiality [10]. A systematic review of LLM-based log analysis [23] notes log drift, hallucination and context and cost limits. These methods establish *that* something is wrong; they do not identify the responsible change or repair it. AION deliberately uses a lightweight, deterministic detector (template masking plus spike detection), because its contribution lies in what happens after detection.

B. AIOps, Incident Management and Root-Cause Analysis

Notaro *et al.* [2] organise AIOps failure-management methods into prevention, detection, diagnosis and remediation, and Zhang *et al.* [22] survey how LLMs are changing these tasks. At Microsoft, Chen *et al.* [1] report two years of incident-management practice, and Ahmed *et al.* [14] evaluate GPT-3.x models on more than 40,000 incidents for recommending root causes and mitigation steps from the incident title and summary. LLM-based RCA systems for cloud incidents [15] and LLM agents for RCA [16] show that diagnosis improves when the model is given relevant context. Service-level RCA methods such as MicroRCA [11] and AutoMAP [12] localise faulty microservices from metrics and dependency graphs [13], and Wittkopp *et al.* [28] progress from anomaly detection towards log labelling and RCA. Integrating LLMs with cloud-native observability for automated RCA and remediation [24] highlights the risks of hallucinated root causes and telemetry that exceeds the model's context. AION adopts the lesson that context quality matters, but replaces free-form retrieval with a bounded, citable evidence pack, and it links the diagnosis to a concrete, testable code change.

TABLE I
Stage coverage of representative work (✓ covered, ○ partial, – not covered)

Work	Det.	Corr.	RCA	Repair	Valid.	Gate
DeepLog, NeuralLog [5], [7]	✓	–	–	–	–	–
MicroRCA, AutoMAP [11], [12]	○	–	✓	–	–	–
LLM incident RCA [14], [15]	–	–	✓	–	–	–
SZZ [25]	–	✓	–	–	–	–
RepairAgent, AutoCodeRover [18], [19]	–	–	○	✓	✓	–
AION (this work)	✓	✓	✓	✓	✓	✓

Det.: detection from production logs; Corr.: correlation with commits and deployments; Valid.: validation in staging before release; Gate: enforced human approval for production.

C. Change Analysis

The SZZ approach of Śliwerski *et al.* [25] links version history to bug reports to identify fix-inducing changes after the fact. AION applies a related intuition online: when an error spike follows a deployment, the commits that touched the failing lines, functions or their dependencies, and that shipped in that deployment, are the strongest suspects.

D. Automated Program Repair and LLM Agents

Le Goues *et al.* [29] review search-, semantics- and learning-based APR. AlphaRepair [30] showed that pre-trained code models can repair bugs zero-shot, and Xia *et al.* [17] found that larger pre-trained models outperform earlier APR tools across five datasets; Zhang *et al.* [31] review the rapidly growing LLM-based APR literature. RepairAgent [18] is an autonomous agent that repaired 164 Defects4J bugs; AutoCodeRover [19] combines program-structure-aware code search with LLMs; SWE-agent [20] designs agent–computer interfaces for software engineering; and SWE-bench [21] evaluates such systems on 2,294 real GitHub issues. Kulsum *et al.* [32] show that patch-validation feedback improves LLM-generated vulnerability fixes. These systems start from an issue or a failing test and end with a candidate patch; none of them observes production, correlates deployments or gates deployment on human approval.

E. Positioning

Table I summarises the stages covered by representative work. AION's novelty is not a better individual stage but the safe, auditable composition of all stages, with deterministic checks around every AI output.

III. System Architecture

Fig. 1 shows the closed loop. A monitored production service emits structured JSON logs. AION, implemented as

a modular monolith, processes them through eight stages; stages 1–6 run automatically in a background pipeline, stage 7 requires a human, and stage 8 runs only after approval. Every transition is appended to an audit trail.

A. Design Principles

- **AI proposes, humans dispose.** The incident lifecycle is a finite-state machine (Fig. 2); no transition leads from AWAITING_APPROVAL to DEPLOYING except through APPROVED, which only the human approval API can enter.
- **Deterministic checks around every AI output.** LLM diagnoses are grounded against evidence; LLM patches pass a code-enforced policy and a validation pipeline whose commands are defined by the operator, never by the model.
- **Explainability.** Every suspect score, grounding warning and validation step is stored with a human-readable reason.
- **Graceful degradation.** Without an LLM, a clearly labelled deterministic analyser reports the top-ranked suspect and proposes a `git revert`.

IV. Methodology

A. Log Normalisation and Signatures

Each log record l is normalised into an event with timestamp, level, message, request context and parsed stack frames. A template $\tau(l)$ is obtained by masking variable fields in a fixed order—UUIDs, URLs, e-mail and IP addresses, hexadecimal identifiers, timestamps and finally numbers—so that *"Order 1042 failed after 3.2ms"* becomes *"Order <NUM> failed after <NUM>ms"*. For error events, the signature is a hash

$$s(l) = h\big(\text{type}(l),\ \text{frames}(l),\ \tau(l)\big), \tag{1}$$

where $\text{frames}(l)$ are the innermost application frames (file and function, deliberately *without* line numbers so that a signature survives unrelated edits). Signatures are stored with occurrence counts and first and last occurrence times.

B. Incident Detection

Let $c_t(s)$ be the number of events with signature s in the current window of length W, and $b(s)$ the average number of occurrences per window over the preceding baseline period B. An incident is opened when

$$c_t(s) \geq m \quad \wedge \quad c_t(s) \geq \rho \cdot b(s), \tag{2}$$

with defaults $W = 60\,\text{s}$, $B = 30\,\text{min}$, $m = 5$ and $\rho = 3$. A brand-new error ($b(s) = 0$) fires once it reaches m, while long-standing low-rate noise does not. Further events with the same signature attach to the open incident, and event time (not wall-clock time) is used, so replayed logs behave identically to live traffic. Events older than the closure of a previous incident with the same signature are ignored, preventing late log lines from reopening a fixed problem.

TABLE II
Change-correlation signals and weights

Signal	Condition	w_k
Blame (line)	commit last modified the failing line	0.40
Blame (function)	commit modified the enclosing function	0.20
Deployment timing	commit shipped in the deployment preceding the errors	0.20
File overlap	commit touched a file in the stack trace	0.15
Dependency overlap	commit touched a module imported by a file in the trace	0.15
Keywords	error identifiers appear in the commit message	0.05

C. Change Correlation

For each commit c in the recent history of the service, AION computes binary or graded signals $f_k(c)$ and combines them into an explainable suspicion score

$$S(c) = \min\Big(1,\ \sum_k w_k\, f_k(c)\Big) \cdot \lambda(c), \qquad c^* = \arg\max_c S(c), \tag{3}$$

with the signals and weights of Table II. Blame signals are computed at the *deployed* revision, function ranges are found with Python's `ast` module, and signals from deeper stack frames are damped by 0.5. Each signal counts once per commit, so a commit touching many frames cannot inflate its score. Commits already running in an earlier deployment before the errors appeared receive $\lambda(c) = 0.5$; otherwise $\lambda(c) = 1$. Ties are broken by recency. $S(c)$ is a ranking heuristic, not a probability; the LLM later reasons over the top candidates and their diffs.

D. Evidence Pack and Grounded Root-Cause Analysis

Instead of placing raw logs in the prompt or retrieving free text [33], AION assembles a bounded *evidence pack* E whose items carry stable identifiers: deduplicated log clusters (`LOG-n`, classified as incident, new or pre-existing noise), a representative trace with failing requests (`TRACE-1`), preceding deployments (`DEPLOY-n`), top suspects with scores, reasons and diffs (`COMMIT-x`), the source of the functions in the trace at the deployed revision (`CODE-n`), and similar past incidents with their fixes (`INC-n`). The LLM must return structured JSON that separates *observations* (each citing an evidence ID) from *inferences* (each citing supporting evidence), together with affected files, a suspected commit, a remediation plan and a confidence value.

A grounding step then checks the response deterministically: observations citing unknown IDs are removed; inferences without valid support are flagged; affected files must exist in the repository; and the suspected commit must be one of the candidates. Any unverifiable claim caps the confidence at 0.60, and a diagnosis with no verifiable observation is capped at 0.30. The exact evidence pack is stored with the report, so a reviewer sees what the model saw. The provider layer is model-neutral: AION supports the Claude API with structured outputs and any OpenAI-compatible endpoint, in-

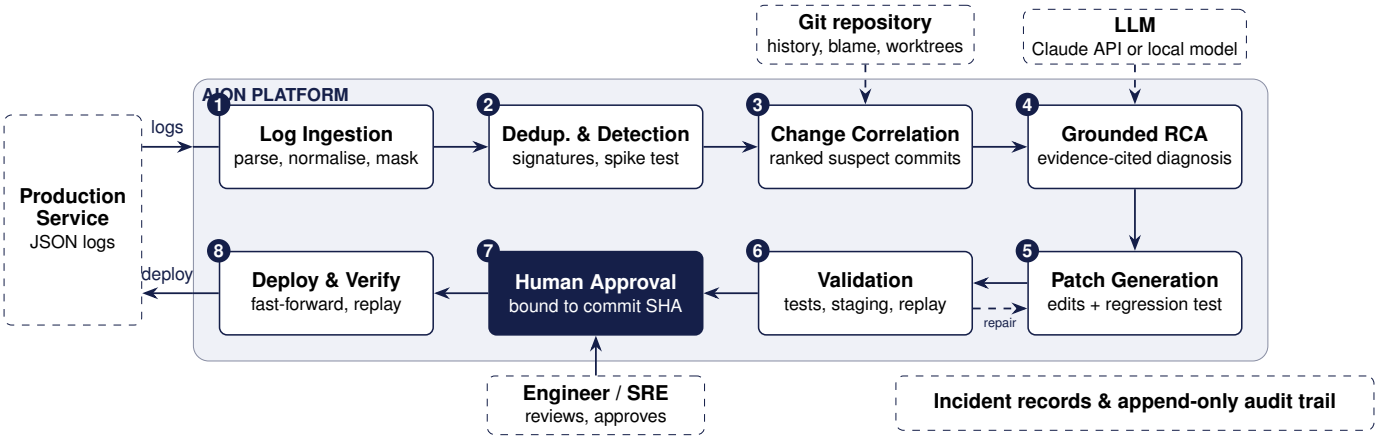


Fig. 1. AION closed-loop architecture. Stages 1–6 run automatically; production deployment (stage 8) is reachable only through an explicit human approval (stage 7). Failed validation loops back to patch generation a bounded number of times.

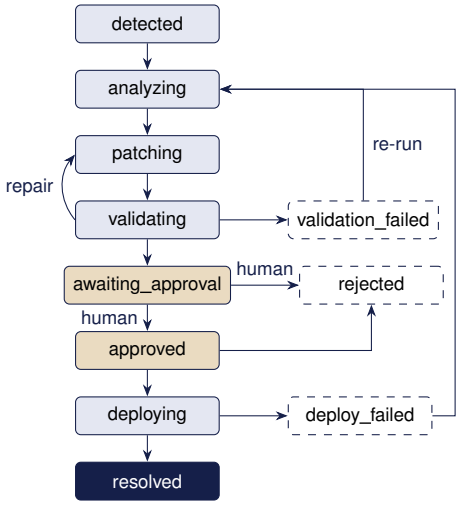


Fig. 2. Incident lifecycle. Shaded states require a human decision; there is no automated edge into APPROVED.

cluding locally hosted models, which keeps sensitive logs on premises.

E. Policy-Constrained Patch Generation

Each patch attempt runs in its own Git worktree and branch. The LLM returns search/replace edits plus a regression test that should reproduce the incident; edits whose search text differs from the source only in indentation are repaired automatically. A code-enforced policy, independent of the prompt, rejects patches that edit tests, CI or dependency files, touch non-existent or unsafe paths, change more than five files or more than 200 lines, or place new tests outside a dedicated name pattern. Algorithm 1 shows the bounded generate–validate–repair loop: validation failures are fed back to the model for a limited number of attempts, after which AION falls back to reverting the top-ranked suspect commit. Both kinds of candidate go through the same validation and the same human gate.

Algorithm 1 Generate–validate–repair with revert fallback

Input: incident I, evidence pack E, diagnosis R, suspects C, attempt budget N
Output: validated candidate P, or escalation to an engineer

$fb \leftarrow \emptyset$
for $i = 1$ **to** N **do**
  $P \leftarrow \text{LLMPATCH}(E, R, fb)$ ▷ edits + regression test
  $P \leftarrow \text{REPAIRINDENTATION}(P)$
  if $\neg\,\text{POLICY}(P)$ **then** $fb \leftarrow$ violation; **continue**
  end if
  $V \leftarrow \text{VALIDATE}(P)$ ▷ Sec. IV-F
  if V.passed **then return** P ▷ to approval gate
  end if
  $fb \leftarrow V$.failures
end for
$P \leftarrow \text{REVERT}(\arg\max_{c \in C} S(c))$
if $\text{VALIDATE}(P)$.passed **then return** P
end if
return $\text{ESCALATE}(I)$ ▷ state VALIDATION_FAILED

F. Deterministic Validation

A patch P is accepted only if

$$V(P) = \text{syntax} \wedge \text{fail2pass} \wedge \text{tests} \wedge \text{staging} \wedge \text{replay}, \quad (4)$$

where *syntax* compiles every changed file; *fail2pass* runs the new regression test against the *unpatched* code (where it must fail, proving it reproduces the incident) and against the patch; *tests* runs the service's full test suite; *staging* starts the patched service from its worktree and waits for its health check; and *replay* re-sends the production requests that failed during the incident (expecting no 5xx response) together with a sample of previously successful requests (expecting unchanged status). All commands come from the operator's service registration and run without a shell and with timeouts. This mirrors the finding that validation feedback is essential for trustworthy LLM fixes [32], but executes it against reproduced production traffic rather than a benchmark test suite.

G. Human Approval, Deployment and Audit

An engineer reviews the evidence, diagnosis, diff and validation report in a web dashboard and approves or rejects the candidate. The approval records the approver, a comment and the commit SHA; deployment is a separate action that rechecks that the approved SHA equals the validated SHA:

$$\text{deploy}(P) \iff V(P) = 1 \ \wedge \ A = \text{approve} \\ \wedge \ \text{sha}(A) = \text{sha}(P). \quad (5)$$

Deployment is a GitOps-style fast-forward of the production branch, followed by verification that the service reports the approved commit and that the previously failing requests now succeed. The incident is marked resolved only after verification; a failed verification returns it for re-investigation. All actions by the pipeline and by humans are written to an append-only audit table.

V. Implementation

AION is implemented in Python with FastAPI, SQLAlchemy and SQLite (write-ahead logging) for persistence, the Git command-line tool for history, blame, worktrees and reverts, and a React dashboard that shows a live pipeline view, evidence, diffs, the approval panel and the audit log. A single background worker executes the pipeline. The LLM layer supports the Claude API through the official SDK and OpenAI-compatible servers such as Ollama. A realistic demonstration service—an orders API with a synthetic multi-commit, multi-deployment Git history—is monitored end to end. The code base is covered by 44 automated tests, including complete end-to-end workflow tests. In a live demonstration on a laptop, an injected fault reached the approval gate about 4 s after the first errors with the deterministic analyser, and approval plus deployment reached a verified, resolved state in about 2 s.

VI. Evaluation

A. Research Questions

RQ1: Does AION detect seeded incidents without false positives? **RQ2:** Does change correlation rank the true culprit first? **RQ3:** Does the LLM diagnosis identify the culprit, and do AI patches pass validation? **RQ4:** Does the safety design stop incorrect patches before the approval gate?

B. Setup

We built four scenarios on the same five-commit base history of the demonstration service, each with a known answer (Table III). The *supplier-feed* scenario is deliberately hard: the culprit commit changes only *data*, so `git blame` on the failing line points at older, innocent code. For each scenario an evaluation harness creates a fresh workspace and database, builds the Git history, executes the real service code to generate normal traffic and trigger requests, ingests the resulting logs through the API and runs the real pipeline up to the approval gate; production deployment is excluded because it always requires a human. We compare two modes: *heuristic* (no LLM: correlation-ranked diagnosis and revert) and *LLM*, using qwen2.5-coder:7b (Q4_K_M quantisation) served locally by Ollama on a laptop GPU with 6 GB of memory, a 12k-token context and temperature 0.1, with at most two AI patch attempts.

TABLE III
Evaluation scenarios (known answers)

Scenario	Fault	What it tests
expired-coupon	`TypeError` on expired coupon	culprit changed the crashing line
supplier-feed	`KeyError: 'price'`	culprit changed only data; blame points at old code
empty-cart-average	`ZeroDivisionError`	latent condition becomes fatal after a new feature
healthy-release	none	false-positive check with background noise

TABLE IV
Final evaluation results

Metric	Heuristic	Local LLM (7B)
Detection correct (incl. no false positive)	4/4	4/4
True culprit ranked first by correlation	3/3	3/3
Diagnosis names true culprit	3/3	2/3
AI patches passing validation	–	0/6
Reached approval gate or correctly silent	4/4	3/4
Incorrect patches reaching the gate	0	0
Mean time per incident	≈5 s	≈100 s
Mean LLM tokens in / out per incident	–	≈13.5k / 1.2k

C. Results

Table IV reports the final results. **RQ1:** both modes detected all three incidents and correctly stayed silent on the healthy release. **RQ2:** the true culprit was ranked first in all three faulty scenarios, including supplier-feed, where it was found through the dependency-overlap signal (the top two scores tied at 0.40 and the tie was broken by recency). **RQ3:** the local LLM explained all three failures in plain language and named the culprit commit in two of three cases (confidence 0.9); in supplier-feed it described the symptom but named no commit. None of the six AI-generated patches passed validation. **RQ4:** every incorrect AI patch was rejected by deterministic checks, and the revert fallback produced validated candidates for two of the three incidents; the third was escalated as VALIDATION_FAILED. No incorrect patch reached the approval gate in any run.

D. Evaluation-Driven Improvements

The evaluation was run five times while the system was improved, and each run exposed a concrete weakness (Table V). In the first run, `git blame` ranked the supplier-feed culprit third and every LLM patch failed the syntax check because of indentation errors. Adding the dependency-overlap signal and restricting the keyword signal to the specific error moved the culprit to rank one in all scenarios; indentation-only repair made AI patches compile and produced regression tests that correctly failed on the buggy code, although the fixes

TABLE V
EVALUATION-DRIVEN IMPROVEMENTS (GATE = REACHED APPROVAL GATE OR CORRECTLY SILENT; RCA = DIAGNOSIS NAMES CULPRIT)

Run	Change since previous run	Heur. gate/RCA	LLM gate/RCA
1	initial design	3/4 · 2/3	1/4 · 2/3
2	dependency signal, scoped keywords	4/4 · 3/3	–
3	indentation-only edit repair	–	1/4 · 2/3
4	three AI attempts instead of two	–	1/4 · 2/3
5	revert fallback after failed AI patches	4/4 · 3/3	3/4 · 2/3

TABLE VI
WHY THE 7B MODEL'S PATCHES FAILED

Scenario	Model behaviour	Caught by
empty-cart-average	correct fix, but regression test used a non-empty order	its own regression test
expired-coupon	guarded one field of `None`, still read another	its own regression test
supplier-feed	priced the product at 0 instead of reading the new field	existing unit tests

themselves remained semantically wrong. A third AI attempt did not help the 7B model, whereas a revert fallback rescued two incidents. All changes were generic and covered by unit tests, rather than tuned to individual scenarios.

E. Failure Analysis of AI Patches

Table VI lists why the local model's patches failed. In empty-cart-average the fix itself was correct, but the model's regression test exercised the wrong order; in expired-coupon the guard was incomplete; in supplier-feed the model "fixed" the crash by silently pricing the product at zero. Each failure was caught by a different deterministic check, which supports the design decision to treat every AI output as untrusted until validated.

VII. DISCUSSION

Diagnosis versus repair. With a small local model, diagnosis was considerably more reliable than repair: the model explained every failure but produced no validated fix. This matches observations that repair quality grows with model size [17] and that agents need validation feedback [18], [32]. Because AION's provider layer is model-neutral, the same harness can measure stronger hosted models with a configuration change.

Safety by construction. The approval guarantee does not depend on model behaviour: it is a property of the state machine, the SHA binding and the operator-defined validation commands. The evaluation shows the value of this separation: every wrong patch was stopped automatically, and every remaining decision went to a human.

Explainable correlation. The weighted score is simple, but every component is a human-readable reason, which helps reviewers and also guides the LLM. The supplier-feed case shows that blame alone is insufficient when the culprit changes data rather than code; the dependency-overlap signal addressed this generically.

A. Limitations and Threats to Validity

The evaluation is small (four self-authored scenarios on one Python service) and each configuration was run once, so LLM variance is not measured; the results characterise behaviour and failure modes rather than general accuracy. Prompt guidance was adjusted after observing failures; to limit overfitting, every change was generic and unit-tested. Detection uses logs only, with fixed thresholds and hand-tuned weights. Staging and production are local processes, request replay covers idempotent requests only, and AI-generated code is validated in a separate process rather than a container sandbox. The current version has no authentication or role-based approvers, and logs may contain sensitive data when an external LLM is used; a local-model option mitigates the latter.

VIII. CONCLUSION AND FUTURE WORK

AION shows that log-based detection, change correlation, LLM diagnosis, patch generation, validation and deployment can be connected in a single closed loop without giving AI the authority to change production. In our evaluation, detection and correlation were accurate and deterministic, a 7B local model was useful for diagnosis but not for repair, and deterministic validation together with a commit-bound human gate stopped every incorrect patch. Future work includes evaluating stronger hosted models with repeated runs, a larger benchmark of seeded incidents, metrics- and trace-based detection, support for further languages, integration with real CI/CD systems and container-based staging, sandboxed validation, secret redaction before prompting, role-based approval, and automatic rollback on failed verification.

REFERENCES

[1] Z. Chen *et al.*, "Towards intelligent incident management: Why we need it and how we make it," in *Proc. 28th ACM Joint Meeting on European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, 2020, pp. 1487–1497.

[2] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 6, pp. 81:1–81:45, 2021.

[3] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40.

[4] Z. Jiang *et al.*, "LILAC: Log parsing using LLMs with adaptive parsing cache," in *Proc. ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2024.

[5] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1285–1298.

[6] W. Meng *et al.*, "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2019, pp. 4739–4745.

[7] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *Proc. 36th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2021, pp. 492–504.

[8] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," arXiv:2103.04475, 2021.

[9] Y. Lee, J. Kim, and P. Kang, "LAnoBERT: System log anomaly detection based on BERT masked language model," arXiv:2111.09564, 2023.

[10] M. De la Cruz Cabello, T. P. Sales, and M. R. Machado, "Log anomaly detection in AIOps: A real-world implementation using large language models," *Systems and Soft Computing*, vol. 8, p. 200475, 2026.

[11] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2020.

[12] M. Ma, J. Xu, Y. Wang, P. Chen, Z. Zhang, and P. Wang, "AutoMAP: Diagnose your microservice-based web applications automatically," in *Proc. The Web Conference (WWW)*, 2020, pp. 246–258.

[13] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022.

[14] T. Ahmed *et al.*, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023.

[15] Y. Chen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th European Conf. Computer Systems (EuroSys)*, 2024.

[16] D. Roy *et al.*, "Exploring LLM-based agents for root cause analysis," in *Companion Proc. 32nd ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2024.

[17] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023.

[18] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE)*, 2025, arXiv:2403.17134.

[19] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2024.

[20] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[21] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024.

[22] L. Zhang, T. Jia, M. Jia *et al.*, "A survey of AIOps in the era of large language models," *ACM Computing Surveys*, 2026.

[23] Z. Ma, J. Yang, and T.-H. Chen, "LLM4Log: A systematic review of large language model-based log analysis," arXiv:2604.16359, 2026.

[24] "Integrating large language models with cloud-native observability for automated root cause analysis and remediation," in *Proc. 3rd Int. Conf. Artificial Intelligence, Systems and Network Security*, 2025.

[25] J. Śliwerski, T. Zimmermann, and A. Zeller, "When do changes induce fixes?" in *Proc. 2nd Int. Workshop on Mining Software Repositories (MSR)*, 2005, pp. 24–28.

[26] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Computing Surveys*, vol. 54, no. 6, pp. 130:1–130:37, 2021.

[27] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE 34th Int. Symp. Software Reliability Engineering (ISSRE)*, 2023.

[28] T. Wittkopp, A. Acker, and O. Kao, "Progressing from anomaly detection to automated log labeling and pioneering root cause analysis," in *Proc. AIOps Workshop, IEEE Int. Conf. Data Mining (ICDM)*, 2023, arXiv:2312.14748.

[29] C. Le Goues, M. Pradel, and A. Roychoudhury, "Automated program repair," *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, 2019.

[30] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proc. 30th ACM ESEC/FSE*, 2022.

[31] Q. Zhang *et al.*, "A systematic literature review on large language models for automated program repair," arXiv:2405.01466, 2024.

[32] U. Kulsum, H. Zhu, B. Xu, and M. d'Amorim, "A case study of LLM for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback," in *Proc. 1st ACM Int. Conf. AI-Powered Software (AIware)*, 2024.

[33] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.